Diseño y Desarrollo de los Servicios Web del Proyecto MONK GRAIN

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

AA5-EV01 construyó un servicio de autenticación como caso de estudio: cuatro endpoints, un módulo aislado, un solo tema. Esta evidencia documenta el otro extremo: la API completa del proyecto formativo, **107 endpoints en 12 grupos**, que es la que sostiene de verdad el sitio de MONK GRAIN.

A esa escala, lo que decide si una API se puede mantener ya no son los endpoints sueltos sino las **reglas transversales**: cómo se nombra, cómo se autoriza, cómo se pagina, qué código de error significa qué. Si cada grupo resuelve esas cosas a su manera, el cliente tiene que aprender doce API en vez de una. Este documento explica esas reglas y, grupo por grupo, la decisión de diseño que no es obvia.

2. Objetivo

Diseñar y codificar las API necesarias para el software del proyecto formativo, con la documentación de cada servicio y control de versiones con Git.

3. Panorama

Grupo	Endpoints	Qué resuelve
sistema	1	Comprobación de vida
autenticación	10	Registro, ingreso, renovación, recuperación
usuarios	7	Perfil, direcciones, consentimientos
catálogo	5	Lo que se puede comprar
carrito	9	La selección antes de pagar
pedidos	6	Lo comprado, con sus sub-órdenes
devoluciones	10	Solicitud, aprobación y seguimiento
reseñas	8	Opiniones de quien compró
oferta y demanda	10	El comprador publica, el caficultor responde
panel del productor	**27**	Lotes, variantes, fotos, pedidos, certificaciones
liquidaciones	7	El dinero de vuelta al vendedor
administración	7	Aprobación, moderación y parámetros
Total	**107**	

El reparto dice algo que ningún diagrama diría: **el panel del productor tiene más endpoints que el catálogo, el carrito y los pedidos juntos**. Es coherente con el negocio. Comprar café es un acto puntual; publicar un lote, corregir el factor de rendimiento, subir las fotos, renovar la certificación y atender los pedidos es trabajo diario. La API refleja dónde está el esfuerzo real.

> El catálogo exhaustivo —los 107 endpoints con su método, su ruta, si exige token y qué hace — está en `ENDPOINTS.md`, junto a este documento. **Ese archivo no se escribe a mano**: lo genera `generar_endpoints.py` desde el documento OpenAPI que publica el servicio en marcha. Un catálogo escrito a mano se desactualiza en la primera semana, cuando alguien añade una ruta y no se acuerda de él.

4. Las reglas transversales

4.1 Nombres

Regla	Sí	No
Sustantivo en plural, no verbo	`GET /orders/{id}`	`GET /getOrder`
El verbo es el método HTTP	`DELETE /cart/items/{id}`	`POST /deleteCartItem`
Minúsculas y guion medio	`/auth/password-reset`	`/auth/passwordReset`
Identificadores UUID	`/orders/{order_id}`	Enteros correlativos

Lo del UUID no es estética. Un identificador correlativo publica cuántos pedidos lleva la plataforma y permite recorrerlos de uno en uno probando suerte; para un mercado que todavía está arrancando, eso es información comercial regalada.

4.2 Versión en la ruta

Todo cuelga de `/api/v1`, con una sola excepción: `/health`. El orquestador que comprueba si el servicio está vivo no debería tener que saber en qué versión va la API, porque entonces una migración a `/v2` tumbaría la comprobación de salud antes que ninguna otra cosa.

4.3 Permisos, no roles

Los endpoints no preguntan «¿eres administrador?» sino «¿tienes este permiso?»:

```
@router.post("/products",
             dependencies=[Depends(require_permissions("product.create"))])
```

Hay **24 permisos** — `product.create`, `order.refund`, `producer.approve`, `settlement.manage`… — y los roles son agrupaciones de ellos. La diferencia se nota el día que hace falta alguien que modere reseñas sin poder tocar precios: con permisos es una fila nueva en una tabla; con roles en duro, hay que tocar cada endpoint.

El error correspondiente es **403 y no 401**: el usuario sí está identificado, lo que le falta es el permiso. Confundirlos hace que el cliente mande a alguien a iniciar sesión cuando ya tiene la sesión abierta.

4.4 El 404 que en realidad es un 403

Cuando alguien pide un pedido que existe pero no es suyo, la API responde **404**, no 403. Un 403 confirmaría que ese pedido existe, y eso ya es información: permitiría recorrer identificadores y contar cuántos pedidos hay. El 404 no distingue «no existe» de «no es tuyo», que es exactamente la ambigüedad que conviene.

4.5 La forma de los errores

FastAPI responde los errores como `{"detail": "..."}`. La API no usa ese formato: un manejador propio los envuelve con un código legible.

```
{"error": {"code":    "business_rule_violation",
           "message": "Este lote se vende desde 250 kg.",
           "details": {"min_qty_kg": 250}}}
```

El `code` existe para que el cliente pueda ramificar sin **analizar el texto del mensaje**, que es lo que acaba haciendo quien solo recibe una cadena, y lo que se rompe en cuanto alguien corrige una tilde. El `details` solo aparece cuando lleva algo accionable dentro: en el ejemplo, el mínimo de kilos, para que el formulario pueda ponerlo en el campo sin leer la frase.

> Este apartado se escribió **después** de que una prueba de AA5-EV04 lo contradijera. La primera versión de este documento daba por hecho el formato de serie de FastAPI —el que uno supone sin mirar—, y la colección esperó un campo `detail` y recibió un `error`. Es el argumento a favor de probar también la documentación, no solo el código: una afirmación plausible y falsa sobrevive a cualquier revisión de lectura.

4.6 Páginas

Toda lista larga se pagina con `?page=1&page_size=20`, y la respuesta trae siempre la misma envoltura:

```
{
  "items":        [ ... ],
  "total":        347,
  "page":         1,
  "page_size":    20,
  "total_pages":  18,
  "has_next":     true,
  "has_previous": false
}
```

`has_next` y `has_previous` parecen redundantes —se pueden calcular con `page` y `total_pages`— y están a propósito: ahorran que cada cliente repita la misma aritmética, y que uno de ellos la haga mal en el último página.

4.7 El dinero y el peso

Dato	Cómo viaja	Por qué
Dinero	Cadena decimal: `"1850000.00"`	Un `float` no representa exactamente 0,1. En pesos colombianos, un redondeo mal hecho sobre una carga de 125 kg se convierte en miles de pesos de diferencia entre lo que el comprador ve y lo que el vendedor cobra
Peso	Kilogramos	La **carga de 125 kg** es una lectura, no una unidad de almacenamiento. Guardar cargas obligaría a representar «media carga y catorce kilos»
Factor de rendimiento	Decimal entre 80 y 130	Gobierna el precio. **Menos es mejor**: el estándar de la Federación es 94, y los filtros acotan por el factor máximo aceptable

4.8 Topes de frecuencia

Solo donde hacen falta, y con dos ceilings distintos en el ingreso:

Endpoint	Tope	Ventana
`/auth/register`	5	5 min
`/auth/verify-email`	10	5 min
`/auth/resend-verification`	3	10 min
`/auth/password-reset`	3	10 min
`/auth/password-reset/confirm`	5	10 min
Subida de imágenes	20	1 min
`/auth/login`	**por cliente y además por cuenta**	

Los dos topes del ingreso no son redundantes, porque cada uno deja pasar lo que el otro para: el de cliente no ve mil clientes ensayando contra un mismo correo; el de cuenta no ve a un cliente recorriendo el padrón de correos. Hacen falta los dos.

5. Los servicios, uno por uno

De cada grupo se anota la decisión que no es evidente. La lista completa de rutas está en `ENDPOINTS.md`.

5.1 Autenticación (10)

El único grupo donde casi todo es público, y por eso el que más protección lleva. Mantiene las mismas decisiones que el módulo Java de AA5-EV01, que se construyó espejando este: mensaje neutral exista o no la cuenta, `202` en el registro para no delatar si se creó algo, y Argon2id para las contraseñas.

`/auth/refresh` y `/auth/logout` aparecen sin cabecera `Authorization` en el catálogo, y **no son públicos**: reciben el refresh token en el cuerpo. Es una distinción que el OpenAPI no sabe expresar y que el catálogo marca aparte, porque leer «público» ahí sería entender lo contrario de lo que pasa.

5.2 Usuarios (7)

Ningún endpoint recibe el identificador del usuario: todos trabajan sobre quien presenta el token. No hay `GET /users/{id}`, y es deliberado — un endpoint así es la vía más corta a una fuga de datos personales, porque basta equivocarse una vez en la comprobación del dueño.

5.3 Catálogo (5)

Público y de solo lectura. Los lotes se crean desde el panel del productor, no desde aquí: separar la lectura pública de la escritura del dueño evita que un fallo de permisos en el camino de lectura —el más visitado y el más cacheable— acabe permitiendo escribir.

El filtro del catálogo tiene dieciséis parámetros, y uno de ellos resume el pivote del proyecto: `max_yield_factor`. No `min`, sino **`max`**, porque en el factor de rendimiento menos es mejor.

5.4 Carrito (9)

El carrito vive en el servidor, no en el navegador. Podría parecer más simple guardarlo en el cliente, pero el precio y la disponibilidad del pergamino cambian, y un carrito que no se entera lleva a la escena peor de todas: el comprador paga un precio que ya no existe.

5.5 Pedidos (6)

Un pedido con lotes de tres fincas genera **tres sub-órdenes** con estado, envío, comisión y liquidación independientes, bajo un único número de pedido. Para el comprador es una compra; para cada vendedor, su propia venta. Sin esa división, el estado «enviado» no significaría nada cuando dos fincas ya despacharon y la tercera no.

5.6 Devoluciones (10)

Los plazos no los inventó el diseño: salen de la revisión legal, en `docs/10`. Diez endpoints para algo que podrían ser tres porque cada paso —solicitar, aprobar, recibir, reembolsar— tiene un actor distinto y deja un rastro que debe poder auditarse después.

5.7 Reseñas (8)

Solo reseña quien compró, y la API lo comprueba contra el pedido, no contra una casilla. Es lo que separa una reseña de un comentario.

5.8 Oferta y demanda (10)

El mercado en su segundo sentido, y el que de verdad distingue a MONK GRAIN de una tienda: el comprador publica lo que busca y los caficultores responden con su precio.

Existe el estado `PARTIALLY_COVERED` porque una solicitud grande se cubre con varios caficultores: un comprador que pide cuatro toneladas rara vez las consigue de una sola finca. Sin ese estado habría que elegir entre decir que la solicitud está abierta —cuando ya está medio cubierta— o cerrada, cuando todavía falta café por conseguir.

El motivo de un rechazo **no es obligatorio**. Fue una decisión explícita, documentada en `docs/15 §5`: obligar a escribir algo produce «no» a secas, que es peor que un rechazo sin motivo, porque parece una explicación y no lo es.

5.9 Panel del productor (27)

El grupo más grande. Lotes, variantes, imágenes, pedidos recibidos y certificaciones.

Las certificaciones tienen fecha de vencimiento y la API la expone, pero **nada la vigila todavía**: no hay proceso en segundo plano. Está anotado en los pendientes del proyecto, y se dice aquí porque un documento de diseño que solo cuenta lo que funciona no sirve para planear lo que falta.

5.10 Liquidaciones (7)

Lo más reciente de la API, y lo que cierra el circuito del dinero: el comprador paga a MONK GRAIN, MONK GRAIN descuenta su comisión del **2,5 %** y liquida al vendedor por periodos, con una retención de cinco días hábiles.

Hasta la auditoría del 1 de noviembre de 2026, la plataforma cobraba y no tenía forma de pagar: el caso de uso más importante para el vendedor no existía en la API. El detalle está en `docs/18`.

5.11 Administración (7)

Aprobación de vendedores, moderación y parámetros. Todo exige el permiso explícito, nunca «ser administrador»: es donde la decisión de §4.3 se paga sola.

6. Documentación generada, no escrita

FastAPI publica el documento OpenAPI del servicio que de verdad está corriendo, con los esquemas de entrada y salida derivados de los modelos de Pydantic. De ahí salen tres cosas sin trabajo manual:

Dónde	Qué es
`/docs`	Swagger UI: se puede probar cada endpoint desde el navegador
`/redoc`	La misma información en formato de lectura
`/openapi.json`	El documento en bruto, del que sale `ENDPOINTS.md`

Y de ahí sale también la colección de Postman de AA5-EV04, importándolo directamente. Es la razón de fondo para cuidar los `summary` y las descripciones en el código: no adornan el código, son la documentación.

7. Lo que falta, dicho aquí

Falta	Por qué importa
Hay esquema sin API detrás: `conversations`, `messages`, `blog_posts` y los cupones	El carrito consume cupones que ningún endpoint crea. Construirlo o retirarlo es una decisión de alcance, no un olvido
No hay proceso en segundo plano	Nada vigila el vencimiento de las certificaciones ni los plazos de las devoluciones
Los tokens viven en `sessionStorage`, no en cookies `httpOnly`	Aceptable para el MVP, no para producción. Migrarlos exige además protección CSRF: es un cambio de arquitectura
El precio no está indexado al precio interno de referencia de la FNC	Fue una decisión, no un olvido: exigía una fuente del precio diario y decidir quién la mantiene

8. Verificación

```
$ curl -s http://localhost:8000/openapi.json | python3 -c "
    import json,sys,collections
    d=json.load(sys.stdin)
    ops=[(m,p) for p,i in d['paths'].items() for m in i
         if m in ('get','post','put','patch','delete')]
    print('TOTAL:', len(ops))"

TOTAL: 107
```

Y las pruebas del backend, que recorren esos endpoints:

```
$ docker compose exec backend pytest -q

453 passed
```

9. Conclusión

Los 107 endpoints están construidos, versionados en Git y documentados en `ENDPOINTS.md`, que se genera desde el servicio en marcha y por tanto no puede mentir sobre lo que existe.

Lo que hace que esta API se pueda sostener no es ninguno de los endpoints, sino las siete reglas que todos obedecen: plural sin verbos, versión en la ruta, permisos en vez de roles, 404 donde un 403 delataría, el error envuelto con un código legible, la misma envoltura en toda lista, y el dinero como cadena decimal. Un cliente que aprende esas siete reglas sabe consumir los 107 endpoints sin leerlos uno por uno — que es, al final, la única medida útil del diseño de una API.